TUGAS 2 IF4020 KRIPTOGRAFI
SEMESTER I TAHUN 2026/2027

BLOCK CIPHER “TENUN-128”



Kelompok 6

Disusun oleh:

Muhammad Aufa Farabi	13523023
Muhammad Dicky Isra	13523075
Muhammad Iqbal Haidar	13523111
Andi Farhan Hidayat	13523128

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

Tugas dan laporan ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Muhammad Aufa Farabi (13523023)



Muhammad Dicky Isra (13523075)



Muhammad Iqbal Haidar (13523111)



Andi Farhan Hidayat (13523128)

I. PENDAHULUAN

A. Latar Belakang

Block cipher adalah algoritma enkripsi simetris yang membagi pesan menjadi blok berukuran tetap dan mengolah setiap blok dengan kunci rahasia yang sama untuk enkripsi dan dekripsi. Algoritma jenis ini dipakai di banyak tempat, misalnya enkripsi berkas, enkripsi disk, dan protokol TLS. Keamanannya bertumpu pada dua prinsip dari Shannon [12], yaitu *confusion* dan *diffusion*. Keduanya biasanya dicapai dengan menyusun transformasi substitusi dan transposisi secara berulang dalam beberapa *round*.

Tugas ini meminta kami merancang *block cipher* baru, mengimplementasikannya bersama lima mode operasi (ECB, CBC, CFB, OFB, dan CTR) tanpa pustaka kriptografi pihak ketiga, lalu mengujinya dengan uji *avalanche effect*, entropi, dan histogram. Rancangan kami bernama **TENUN-128**. Nama ini diambil dari kain tenun, yang dibuat dengan menyusun benang secara berulang.

Selain algoritmanya, kami membuat program yang dapat dijalankan lewat antarmuka baris perintah (CLI) maupun antarmuka terminal interaktif (TUI). Program ini memakai *padding* PKCS#7 dan memverifikasi integritas data dengan CMAC yang dibangun di atas TENUN-128 sendiri. Dengan begitu, berkas yang sudah dienkripsi tidak hanya terjaga kerahasiaannya, tetapi perubahan pada berkas atau penggunaan kunci yang salah juga akan terdeteksi saat dekripsi.

B. Perbandingan dengan *Block Cipher* Sejenis

Tabel 1 membandingkan TENUN-128 dengan beberapa *block cipher* yang kami jadikan rujukan. Parameter cipher pembanding diambil dari spesifikasi masing-masing [13, 2, 6, 14, 4, 9].

Tabel 1. Perbandingan TENUN-128 dengan *block cipher* sejenis.

Cipher	Blok (bit)	Kunci (bit)	*Round*	Struktur	Ciri utama
DES [13]	64	56	16	Feistel	*S-box* 6→4 bit dan permutasi bit; kunci terlalu pendek
AES [2]	128	128/192/256	10/12/14	SPN	*S-box* 8 bit dari invers GF(2^8), *ShiftRows*, *MixColumns*
PRESENT [6]	64	80/128	31	SPN	*S-box* 4 bit dan permutasi bit; sangat ringan
Serpent [14]	128	128/192/256	32	SPN	delapan *S-box* 4 bit dan transformasi linear
Twofish [4]	128	128/192/256	16	Feistel	*S-box* bergantung kunci, matriks MDS, PHT
TEA [9]	64	128	64	Feistel	hanya geser, XOR, dan tambah; konstanta δ = `9E3779B9`
TENUN-128	**128**	**128**	**16 + 1**	**SPN**	*S-box* 4 bit, permutasi *nibble*, penjumlahan modulo, rotasi *word* bergantung kunci

DES memakai jaringan Feistel dengan kunci efektif 56 bit, sehingga kini mudah dipecahkan dengan pencarian kunci menyeluruh (*brute force*). AES mengganti Feistel dengan jaringan substitusi-permutasi (SPN) pada blok 128 bit dan menjadi standar yang dipakai luas. PRESENT dan Serpent memakai *S-box* 4 bit yang dipilih dengan kriteria diferensial dan linear yang ketat. Twofish dan TEA menunjukkan bahwa penjumlahan modulo dan rotasi dapat dipakai sebagai sumber nonlinearitas dan *diffusion* yang murah.

C. Gagasan Utama Rancangan

TENUN-128 adalah SPN dengan blok 128 bit, kunci 128 bit, dan 16 *round* ditambah satu *whitening* di akhir, sehingga ada 17 *round key*. *State* disusun sebagai matriks 4×4 byte seperti pada AES. Ide yang membedakan TENUN-128 adalah gabungan beberapa komponen sederhana dalam satu fungsi *round*:

1. *S-box* 4 bit yang diterapkan pada setiap *nibble*, seperti pada PRESENT dan Serpent. Tabelnya kecil sehingga mudah dianalisis, dan kami memilihnya lewat pencarian dengan kriteria DU = LU = 4 serta tanpa *fixed point*.
2. Permutasi pada tingkat *nibble*, $j = (9i + 5) \bmod 32$, untuk menyebarkan keluaran *S-box* ke seluruh *state*.
3. Rantai penjumlahan modulo 2^8 pada setiap kolom, dengan gagasan dari penjumlahan modulo pada Twofish dan operasi ARX. Operasi ini nonlinear terhadap XOR karena adanya *carry*, dan sekaligus mencampur byte dalam satu kolom.
4. Rotasi *word* 32 bit pada setiap baris, dengan jumlah geser yang ditentukan oleh *round key*. Rotasi ini memindahkan bit antarkolom sehingga penjumlahan pada *round* berikutnya mencampur seluruh *state*.
5. *Key Schedule* yang memakai konstanta dari rasio emas, `9E3779B9`, seperti pada TEA [9]. Konstanta ini membuat *round key* tetap berbeda satu sama lain walaupun kunci utamanya lemah, misalnya semua nol.

Di luar algoritmanya, program menyimpan hasil enkripsi dalam format berkas tersendiri (*envelope*) yang memuat mode, IV, dan *tag* CMAC. Dekripsi memeriksa *tag* lebih dulu sebelum menghasilkan *plaintext*.

D. Sistematika Laporan

Bab II berisi dasar teori. Bab III menjelaskan rancangan TENUN-128. Bab IV menjelaskan implementasi program serta hasil uji enkripsi dan dekripsi, *padding*, dan integritas. Bab V membahas analisis keamanan. Bab VI berisi kesimpulan dan saran, lalu diikuti daftar pustaka dan lampiran.

II. DASAR TEORI

A. *Block Cipher*, *Confusion*, dan *Diffusion*

Block cipher dengan blok n bit dan kunci K adalah fungsi $E_K : \{0,1\}^n \to \{0,1\}^n$ yang merupakan permutasi, dan inversnya $D_K = E_K^{-1}$ dipakai untuk dekripsi. Menurut Shannon [12], cipher yang baik harus memiliki dua sifat. Pertama, *confusion*, yaitu hubungan antara kunci dan *ciphertext* dibuat serumit mungkin. Kedua, *diffusion*, yaitu pengaruh satu bit *plaintext* atau satu bit kunci disebarkan ke banyak bit *ciphertext*. Kedua sifat ini umumnya dicapai dengan menyusun substitusi (yang nonlinear) dan transposisi atau pencampuran (yang linear) berulang kali dalam *iterated cipher*, dengan *round key* berbeda pada tiap *round*.

Ada dua struktur utama. Pada jaringan Feistel (DES, Twofish), blok dibagi dua dan fungsi *round* tidak harus dapat dibalik. Pada jaringan substitusi-permutasi atau SPN (AES, PRESENT, Serpent), seluruh blok melewati lapisan substitusi dan permutasi, sehingga setiap lapisan harus bijektif agar dapat didekripsi. TENUN-128 adalah SPN.

B. *S-box* dan Kriteria Kekuatannya

S-box 4 bit adalah tabel substitusi $S : \{0,1\}^4 \to \{0,1\}^4$ dengan 16 entri. Agar dapat dibalik, S harus bijektif. Ketahanannya terhadap kriptanalisis diferensial [19] dan linear [20] dinilai dengan dua tabel berikut.

Differential Distribution Table (DDT):

$$\mathrm{DDT}[\Delta x][\Delta y] = \Big|\{x : S(x) \oplus S(x \oplus \Delta x) = \Delta y\}\Big|, \qquad \mathrm{DU}(S) = \max_{\Delta x \neq 0, \Delta y} \mathrm{DDT}[\Delta x][\Delta y]. \tag{1}$$

Linear Approximation Table (LAT):

$$\mathrm{LAT}[a][b] = \Big|\{x : a \cdot x = b \cdot S(x)\}\Big| - 8, \qquad \mathrm{LU}(S) = \max_{a \neq 0, b \neq 0} \Big|\mathrm{LAT}[a][b]\Big|. \tag{2}$$

Untuk *S-box* 4 bit yang bijektif, nilai terbaik yang dapat dicapai adalah DU = 4 dan LU = 4, seperti pada *S-box* PRESENT [6]. Kriteria tambahan yang lazim adalah tidak adanya *fixed point*, yaitu $S(x) \neq x$ untuk semua x.

C. Operasi XOR, Penjumlahan Modulo, dan Rotasi

Penjumlahan modulo 2^8 bersifat nonlinear terhadap XOR karena *carry* merambat dari bit rendah ke bit tinggi. Akibatnya, perubahan satu bit masukan dapat mengubah beberapa bit keluaran. Rotasi memindahkan bit antarposisi tanpa menghilangkan informasi dan murah dihitung. Kombinasi penjumlahan, rotasi, dan XOR (disebut ARX) dipakai pada banyak cipher, termasuk Twofish, RC6, dan HIGHT [4, 3, 5].

D. Mode Operasi

Block cipher hanya mengenkripsi satu blok. Mode operasi menentukan cara memproses pesan yang lebih panjang. Tabel 2 merangkum lima mode yang diminta tugas, dengan notasi dari NIST SP 800-38A [15]. P_i dan C_i adalah blok *plaintext* dan *ciphertext* ke-i, E_K adalah enkripsi satu blok, dan IV adalah *initialization vector* (pada CTR, nilai awal *counter*).

Tabel 2. Persamaan mode operasi yang diimplementasikan.

Mode	Enkripsi	Dekripsi	*Padding*
ECB	$C_i = E_K(P_i)$	$P_i = D_K(C_i)$	ya
CBC	$C_i = E_K(P_i \oplus C_{i-1})$, $C_0 = IV$	$P_i = D_K(C_i) \oplus C_{i-1}$	ya
CFB	$C_i = P_i \oplus E_K(C_{i-1})$, $C_0 = IV$	$P_i = C_i \oplus E_K(C_{i-1})$	tidak
OFB	$O_i = E_K(O_{i-1})$, $O_0 = IV$; $C_i = P_i \oplus O_i$	$P_i = C_i \oplus O_i$	tidak
CTR	$C_i = P_i \oplus E_K(IV + i - 1)$	$P_i = C_i \oplus E_K(IV + i - 1)$	tidak

ECB dan CBC mengharuskan data berukuran kelipatan blok, sehingga memerlukan *padding*. PKCS#7 menambahkan N byte bernilai N ($1 \leq N \leq 16$) di akhir pesan [17]. *Padding* selalu ditambahkan, bahkan ketika panjang pesan sudah kelipatan blok, supaya pelepasannya tidak ambigu. CFB, OFB, dan CTR membangkitkan *keystream* yang di-XOR dengan data, sehingga blok terakhir yang pendek cukup dipotong dan tidak perlu *padding*.

E. Integritas Data dan CMAC

Enkripsi saja tidak menjamin integritas. Pada CTR, misalnya, membalik satu bit *ciphertext* akan membalik bit yang sama pada *plaintext* tanpa terdeteksi. Karena itu diperlukan *message authentication code* (MAC). CMAC [16] adalah MAC berbasis *block cipher*. Pesan diproses seperti CBC-MAC, dan blok terakhir di-XOR dengan subkunci K_1 (bila blok lengkap) atau K_2 (bila blok diberi *padding*). Subkunci dihitung sebagai $K_1 = 2 \cdot E_K(0^{128})$ dan $K_2 = 2 \cdot K_1$ pada $GF(2^{128})$ dengan polinomial $x^{128} + x^7 + x^2 + x + 1$. Pola *encrypt-then-MAC*, yaitu *tag* dihitung dari *ciphertext* dan diverifikasi sebelum dekripsi, terbukti aman secara umum bila cipher dan MAC-nya aman dan keduanya memakai kunci yang berbeda [18].

F. Metrik Evaluasi

Avalanche effect: perubahan satu bit masukan seharusnya mengubah sekitar separuh bit keluaran [21, 22]. Nilainya dihitung sebagai jarak Hamming antara dua *ciphertext* dibagi jumlah bit. Untuk permutasi acak yang ideal pada blok n bit, jumlah bit yang berubah berdistribusi binomial dengan rata-rata n/2 dan simpangan baku $\sqrt{n}/2$.

Entropi Shannon dari byte *ciphertext* dihitung sebagai $H = -\sum_{v=0}^{255} p_v \log_2 p_v$ dan bernilai maksimum 8 bit/byte untuk distribusi seragam. Histogram menampilkan frekuensi tiap nilai byte, dan *ciphertext* yang baik memiliki histogram yang mendekati seragam.

G. *Related Works*

AES [2] dan Serpent [14] adalah SPN 128 bit dengan *S-box* dan lapisan linear yang telah dianalisis secara luas. PRESENT [6] menekankan *S-box* 4 bit dan permutasi bit untuk perangkat yang sangat terbatas. Twofish [4] dan MARS [11] memakai penjumlahan modulo dan rotasi di dalam fungsi *round*, RC6 [3] memakai rotasi bergantung data, dan TEA [9] memakai konstanta δ dari rasio emas. Cipher ringan seperti HIGHT [5], KLEIN [7], CLEFIA [8], dan WARP [10] menunjukkan variasi rancangan berbasis *nibble* dan byte. Tutorial Heys [1] menjadi acuan umum dalam menyusun komponen cipher. TENUN-128 tidak menyalin cipher mana pun, tetapi memakai gagasan umum dari cipher-cipher tersebut dan menggabungkannya menjadi fungsi *round* yang baru.

III. PERANCANGAN *BLOCK CIPHER*

A. Parameter

Tabel 3 merangkum parameter TENUN-128. Operasi dilakukan pada tingkat *nibble* (4 bit), byte, dan *word* 32 bit.

Tabel 3. Parameter TENUN-128.

Parameter	Nilai
Ukuran blok	128 bit (16 byte)
Panjang kunci utama	128 bit (16 byte), sama dengan ukuran blok
Struktur	jaringan substitusi-permutasi (SPN), bukan Feistel
Jumlah *round*	16 *round* ditambah 1 *whitening* di akhir (17 *round key*, $K_1, \ldots, K_{17}$)
Representasi *state*	matriks 4×4 byte yang diisi per baris, atau 32 *nibble*
Operasi tiap *round*	*Key Addition* (XOR kunci), *Substitution* (*S-box*), *Permutation* (permutasi *nibble*), *Mixing* (penjumlahan modulo), *Rotation* (rotasi *word*)
Pembangkitan *round key*	*Key Schedule*, dengan konstanta $r \cdot$ `9E3779B9` mod 2^{32}
Teknik yang diwajibkan tugas	substitusi *S-box*, transposisi, serta penjumlahan modulo dan rotasi sebagai operasi tambahan

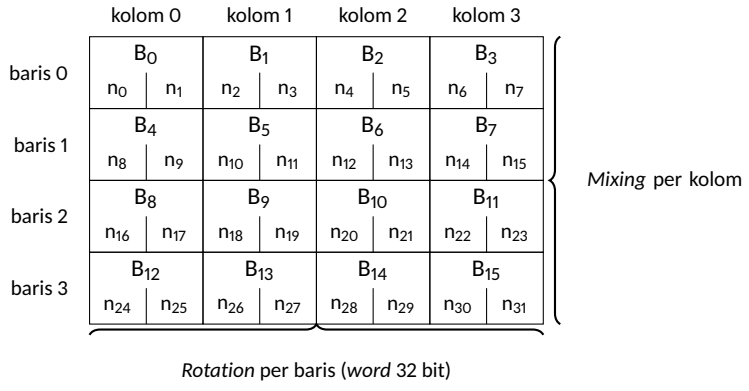
Tabel 4 memetakan setiap langkah ke fungsi yang mengimplementasikannya.

Tabel 4. Pemetaan langkah ke fungsi pada implementasi.

Langkah	Fungsi di kode
Key Addition	`TENUN128Cipher.XOR_round_key`
Substitution	`SBox.substitute_state`
Permutation	`Permutation.permute_state`
Mixing	`TENUN128Cipher.modulo_opr`
Rotation	`TENUN128Cipher.rotate_opr`
Key Schedule	`KeyExpansion.generate_round_keys`

B. Representasi *State*

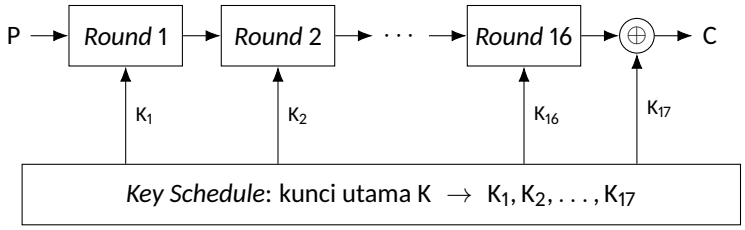
Blok 16 byte $B_0 B_1 \cdots B_{15}$ diisikan *per baris* ke matriks 4×4: byte B_p berada pada baris $\lfloor p/4 \rfloor$ dan kolom $p \bmod 4$ (Gambar 1). Setiap byte terdiri atas dua *nibble*, yaitu n_{2p} (tinggi) dan n_{2p+1} (rendah), sehingga *state* juga dapat dipandang sebagai 32 *nibble* berindeks 0 sampai 31. *Mixing* bekerja pada kolom (byte $B_c, B_{4+c}, B_{8+c}, B_{12+c}$), sedangkan *Rotation* bekerja pada baris yang dibaca sebagai *word* 32 bit dengan urutan *big-endian*.



Gambar 1. Tata letak *state* 4×4 byte (diisi per baris) beserta indeks byte B_p dan *nibble* n_i.

C. Alur Enkripsi

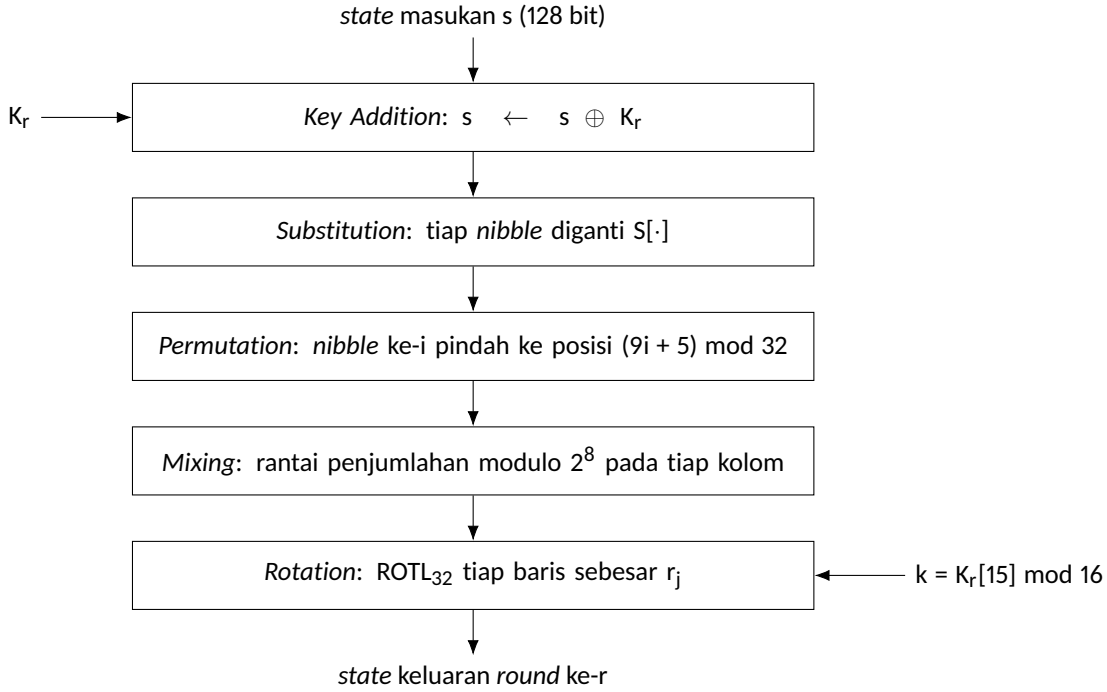
Enkripsi menjalankan 16 *round* dengan struktur yang sama, lalu meng-XOR hasilnya dengan K_{17} (Gambar 2). *Key Schedule* membangkitkan $K_1, \ldots, K_{17}$ dari kunci utama (Subbab F).



Gambar 2. Alur enkripsi TENUN-128: 16 *round* dan *whitening* di akhir.

D. Fungsi *Round*

Untuk r = 1, . . . , 16 dengan *state* s, setiap *round* terdiri atas lima langkah berurutan (Gambar 3).



Gambar 3. Satu *round* TENUN-128.

1. *Key Addition*

$$s \leftarrow s \oplus K_r.$$

State di-XOR dengan *round key* sehingga setiap langkah berikutnya bergantung pada kunci. XOR adalah involusi, jadi langkah ini dibalik dengan operasi yang sama.

2. *Substitution* (*S-box*)

Setiap byte b dipecah menjadi dua *nibble*, lalu masing-masing diganti dengan nilai dari *S-box* 4 bit S (Tabel 5):

$$b' = \big(S[b \gg 4] \ll 4\big) \mid S[b \,\&\, \texttt{0F}].$$

S-box dibangkitkan lewat pencarian terarah (Subbab G). Karena S bijektif, inversnya dibentuk dengan $S^{-1}[S(x)] = x$. *Substitution* adalah sumber utama *confusion*.

Tabel 5. *S-box* S dan inversnya (heksadesimal).

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
S(x)	A	9	B	4	5	E	3	D	C	F	6	0	7	2	1	8
y	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$S^{-1}(y)$	B	E	D	6	3	4	A	C	F	1	0	2	8	7	5	9

3. *Permutation* (pada *nibble*)

State dipandang sebagai 32 *nibble* $n_0, \ldots, n_{31}$ (Gambar 1). *Nibble* pada posisi i dipindahkan ke posisi

$$j = (9i + 5) \bmod 32, \qquad i = 0, \ldots, 31.$$

Karena $\gcd(9, 32) = 1$, pemetaan ini bijektif (Tabel 6). Dari tabel terlihat bahwa dua *nibble* dari satu byte selalu pindah ke dua byte berbeda yang indeksnya berselisih 5 (modulo 16), yaitu pada kolom yang bersebelahan. Delapan *nibble* dari satu baris tersebar ke keempat kolom, masing-masing dua. Sebaliknya, delapan *nibble* dari satu kolom tersebar ke keempat baris tetapi hanya ke dua kolom, masing-masing empat. Pencampuran antarkolom yang penuh karena itu bergantung pada *Rotation* dan *round* berikutnya.

Tabel 6. Pemetaan *Permutation*: *nibble* pada posisi i pindah ke posisi j.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
j	5	14	23	0	9	18	27	4	13	22	31	8	17	26	3	12
i	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
j	21	30	7	16	25	2	11	20	29	6	15	24	1	10	19	28

4. *Mixing* (penjumlahan modulo)

Untuk setiap kolom $c = 0, \ldots, 3$, misalkan (a, b, c, d) adalah empat byte kolom itu dari baris 0 sampai 3. Nilai barunya dihitung berantai:

$$a' = (a + d) \bmod 2^8, \qquad b' = (b + a') \bmod 2^8,$$
$$c' = (c + b') \bmod 2^8, \qquad d' = (d + c') \bmod 2^8.$$

Dalam bentuk matriks bilangan bulat, pemetaan ini adalah

$$\begin{pmatrix} a' \\ b' \\ c' \\ d' \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 2 \end{pmatrix}}_{M} \begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix} \pmod{2^8}, \qquad \det M = 1.$$

Karena $\det M = 1$, M memiliki invers bulat sehingga *Mixing* bijektif pada $\mathbb{Z}_{256}^4$. Perubahan pada a atau d memengaruhi keempat keluaran, perubahan pada b memengaruhi b', c', dan d', dan perubahan pada c memengaruhi c' dan d'. Karena penjumlahan merambatkan *carry* ke bit yang lebih tinggi, *Mixing* juga memberi nonlinearitas terhadap XOR dan menyebarkan pengaruh bit rendah ke bit tinggi dalam satu byte.

5. *Rotation* (rotasi bergantung kunci)

Baris ke-j (j = 0, . . . , 3) dibaca sebagai *word* 32 bit *big-endian* W_j. Dari *round key* diambil *nibble* rendah byte terakhir, $k = K_r[15] \bmod 16$, lalu

$$r_j = (2k + 1 + 8j) \bmod 32, \qquad W'_j = \mathrm{ROTL}_{32}(W_j, r_j),$$
$$\mathrm{ROTL}_{32}(x, n) = \big((x \ll n) \mid (x \gg (32 - n))\big) \,\&\, \mathtt{FFFFFFFF}.$$

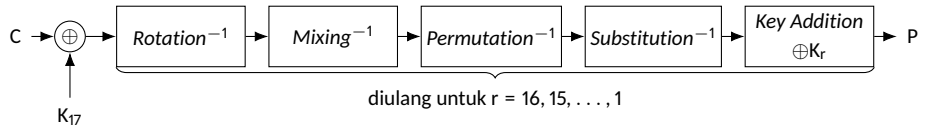
Nilai $2k+1$ selalu ganjil dan $8j \in \{0, 8, 16, 24\}$ selalu genap, sehingga semua r_j ganjil (tidak pernah 0) dan keempatnya selalu berbeda. Tiap baris diputar dengan jumlah yang berbeda, jadi bit-bit dari satu kolom pada baris-baris yang berlainan akan jatuh ke kolom yang berbeda pada *round* berikutnya. Langkah ini melengkapi *Mixing* yang hanya bekerja di dalam kolom. Terdapat 16 kemungkinan himpunan $(r_0, \ldots, r_3)$ yang dipilih oleh *round key*.

Setelah *round* ke-16, *state* di-XOR dengan kunci terakhir: $C = s \oplus K_{17}$.

E. Dekripsi

Dekripsi membalik setiap langkah dengan urutan terbalik, dan memakai *round key* dari K_{17} sampai K_1 (Gambar 4). Pseudocode-nya sebagai berikut.

```
s <- C xor K17
untuk r = 16 turun ke 1:
    1. Rotation^-1     : W_j <- ROTR32(W_j, r_j)    (r_j dari K_r yang sama)
    2. Mixing^-1       : d <- (d' - c') mod 256
                         c <- (c' - b') mod 256
                         b <- (b' - a') mod 256
                         a <- (a' - d ) mod 256
    3. Permutation^-1  : nibble pada posisi j kembali ke i = 25 (j - 5) mod 32
    4. Substitution^-1 : tiap nibble diganti dengan S^-1
    5. Key Addition    : s <- s xor K_r
P <- s
```



Gambar 4. Alur dekripsi TENUN-128.

Balikan setiap langkah dijelaskan berikut ini.

- *Rotation*$^{-1}$: rotasi ke kanan dengan r_j yang sama (dihitung dari K_r), yaitu $\mathrm{ROTR}_{32}(x, n) = (x \gg n) \mid (x \ll (32 - n))$ pada 32 bit.
- *Mixing*$^{-1}$: pengurangan harus dilakukan berurutan $d \to c \to b \to a$. Nilai $a = a' - d$ memerlukan

d yang asli, bukan d', jadi d harus dipulihkan lebih dulu. Urutan lain menghasilkan keluaran yang salah.

- *Permutation*$^{-1}$: dari $j = 9i + 5 \pmod{32}$ diperoleh $i = 9^{-1}(j - 5) = 25(j - 5) \bmod 32$, karena $9 \times 25 = 225 \equiv 1 \pmod{32}$.
- *Substitution*$^{-1}$: substitusi dengan S^{-1} (Tabel 5).
- *Key Addition*: XOR dengan kunci yang sama mengembalikan *state* semula.

F. *Key Schedule*

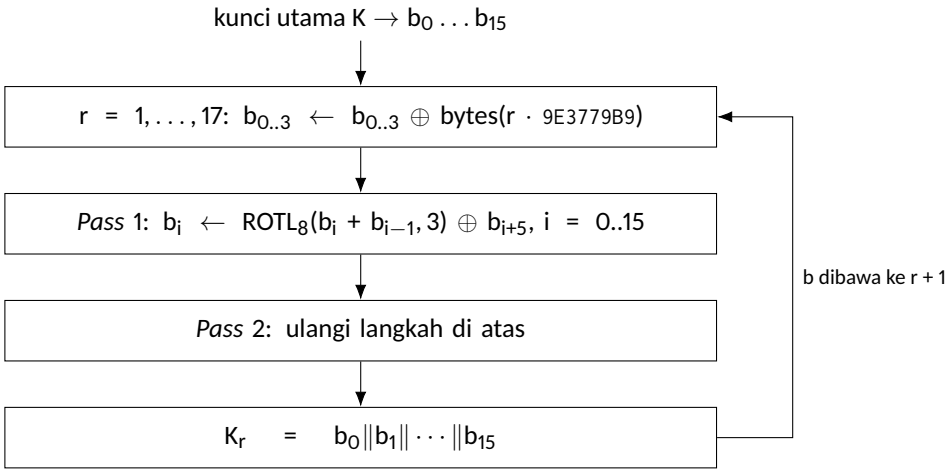
Kunci utama 16 byte disusun sebagai larik melingkar $b_0, \ldots, b_{15}$ (indeks dihitung modulo 16). Larik b tidak dikembalikan ke kunci utama di antara dua *round*. Ia terus berubah, sehingga K_r merupakan turunan dari K_{r-1}. Untuk $r = 1, \ldots, 17$:

1. $(b_0, b_1, b_2, b_3) \leftarrow (b_0, b_1, b_2, b_3) \oplus \text{bytes}(C_r)$, dengan konstanta publik $C_r = r \cdot \texttt{9E3779B9} \bmod 2^{32}$ yang dipecah menjadi 4 byte *big-endian*.
2. Ulangi dua kali, untuk $i = 0, \ldots, 15$ secara berurutan (nilai b_i yang baru langsung dipakai oleh langkah berikutnya):

$$b_i \leftarrow \text{ROTL}_8\big((b_i + b_{i-1}) \bmod 2^8,\ 3\big) \ \oplus\ b_{(i+5) \bmod 16}.$$

3. $K_r = b_0 \,\|\, b_1 \,\|\, \cdots \,\|\, b_{15}$.

ROTL_8 adalah rotasi kiri 3 posisi pada satu byte (8 bit), berbeda dari ROTL_{32} pada *Rotation*. Konstanta C_r bersifat publik, sesuai prinsip Kerckhoffs, dan berasal dari rasio emas seperti pada TEA [9]. Konstanta ini membedakan setiap *round* sehingga *round key* tidak seragam walaupun kunci utamanya lemah, misalnya semua nol, dan mempersulit *slide attack*. *round key* untuk contoh uji ada di Lampiran.



Gambar 5. Alur *Key Schedule*.

G. Pembangkitan *S-box*

S-box tidak diambil dari cipher lain dan tidak dibuat dengan mengacak tabel begitu saja. Skrip `tools/sboxGenerator.py` mengacak permutasi dari $\{0, \ldots, 15\}$ dengan `random.Random(seed)` dan hanya menerima kandidat yang memenuhi semua syarat berikut:

1. bijektif (sudah pasti, karena kandidat adalah permutasi),
2. DU = 4, yaitu nilai terbesar pada DDT untuk $\Delta x \neq 0$,

3. LU = 4, yaitu nilai mutlak terbesar pada LAT untuk a, b $\neq 0$,
4. tidak ada *fixed point*, yaitu S(x) $\neq$ x untuk semua x.

Jika ada syarat yang tidak terpenuhi, kandidat baru dibuat. Dengan *seed* 42 (agar dapat direproduksi), kandidat diterima pada percobaan ke-2. *S-box* yang dipakai pada implementasi adalah

$$S = [\texttt{A}, 9, \texttt{B}, 4, 5, \texttt{E}, 3, \texttt{D}, \texttt{C}, \texttt{F}, 6, \texttt{0}, 7, 2, 1, 8].$$

Dengan fungsi pada `tools/sboxGenerator.py`, kami memeriksa bahwa DU = 4, LU = 4 (nonlinearitas $8 - \text{LU} = 4$), dan jumlah *fixed point* = 0. DDT dan LAT lengkap serta cara menghitungnya ada di Lampiran.

H. Contoh Satu *Round*

Sebagai contoh, kunci utama K = `000102030405060708090A0B0C0D0E0F` dan *plaintext* P = "LOREM IPSUM SIAM" = `4C4F52454D20495053554D205349414D` (16 byte). *round key* pertama adalah K_1 = `41CB7FBEDEB6E522E4F919B452AD0F47`. Gambar 6 menunjukkan *state* setelah setiap langkah pada *round* pertama (nilai heksadesimal, disusun per baris).

Masukan P

$$\begin{bmatrix} 4C\ 4F\ 52\ 45 \\ 4D\ 20\ 49\ 50 \\ 53\ 55\ 4D\ 20 \\ 53\ 49\ 41\ 4D \end{bmatrix}$$

Setelah *Key Addition*

$$\begin{bmatrix} 0D\ 84\ 2D\ FB \\ 93\ 96\ AC\ 72 \\ B7\ AC\ 54\ 94 \\ 01\ E4\ 4E\ 0A \end{bmatrix}$$

Setelah *Substitution*

$$\begin{bmatrix} A2\ C5\ B2\ 80 \\ F4\ F3\ 67\ DB \\ 0D\ 67\ E5\ F5 \\ A9\ 15\ 51\ A6 \end{bmatrix}$$

Setelah *Permutation*

$$\begin{bmatrix} 55\ 5D\ 0A\ 96 \\ 3B\ 1F\ BF\ 21 \\ 76\ 2A\ 50\ 4C \\ 5E\ 78\ 6A\ DF \end{bmatrix}$$

Setelah *Mixing*

$$\begin{bmatrix} B3\ D5\ 74\ 75 \\ EE\ F4\ 33\ 96 \\ 64\ 1E\ 83\ E2 \\ C2\ 96\ ED\ C1 \end{bmatrix}$$

Setelah *Rotation*

$$\begin{bmatrix} BA\ 3A\ D9\ EA \\ CB\ 77\ 7A\ 19 \\ 32\ 0F\ 41\ F1 \\ 4B\ 76\ E0\ E1 \end{bmatrix}$$

Gambar 6. *State* pada *round* pertama untuk K = `000102...0F` dan P = "LOREM IPSUM SIAM".

Perhitungan rinci pada *round* ini adalah sebagai berikut.

- *Substitution*: byte pertama setelah *Key Addition* adalah `0D`, dengan *nibble* `0` dan `D`. Karena S[`0`] = `A` dan S[`D`] = `2`, byte tersebut menjadi `A2`.
- *Permutation*: *nibble* n_1 = `2` dipindahkan ke posisi $(9 \cdot 1 + 5) \bmod 32 = 14$, sehingga n_{14} = `2` setelah permutasi.
- *Mixing*, kolom 0: (a, b, c, d) = (`55`, `3B`, `76`, `5E`) menghasilkan a' = `55` + `5E` = `B3`, b' = `3B` + `B3` = `EE`, c' = `76` + `EE` = `64`, dan d' = `5E` + `64` = `C2` (semuanya modulo 2^8).
- *Rotation*: k = K_1[15] mod 16 = 7, sehingga $(r_0, r_1, r_2, r_3) = (15, 23, 31, 7)$. Baris 0 bernilai `B3D57475` dan menjadi `BA3AD9EA` setelah ROTL_{32} sebesar 15.

Setelah 16 *round* dan *whitening*, *ciphertext* untuk blok ini adalah C = `43D34D50CEC3A30EFC2CC3D8C50573FE`. Dekripsi C dengan kunci yang sama mengembalikan *plaintext* semula. Potongan kode untuk mereproduksi contoh ini, beserta vektor uji lainnya, ada di Lampiran.

I. Alasan Rancangan

Untuk *confusion*, hubungan antara kunci dan *ciphertext* dikaburkan oleh tiga hal. Pertama, *S-box* 4 bit dengan DU = LU = 4 yang diterapkan pada 32 *nibble* di setiap *round*. Kedua, nonlinearitas penjumlahan modulo pada *Mixing*. Ketiga, *Key Schedule* yang membuat setiap *round key* bergantung pada seluruh bit kunci utama melalui dua *pass* penjumlahan, rotasi, dan XOR.

Untuk *diffusion*, pengaruh satu bit disebarkan secara bertahap. *Substitution* menyebarkan perubahan ke seluruh bit dalam *nibble*. *Permutation* memindahkan *nibble* yang berubah ke byte dan kolom lain. *Mixing* mencampur byte dalam satu kolom dan menyebarkan bit rendah ke bit tinggi. *Rotation* memindahkan bit antarkolom sehingga *Mixing* pada *round* berikutnya mencampur kolom yang lain. Pada tingkat blok, perubahan satu bit *plaintext* maupun satu bit kunci mengubah sekitar 50% bit *ciphertext* setelah 16 *round* (Bab V).

Kelemahan dan pertanyaan yang belum terjawab:

- Belum ada analisis diferensial dan linear secara formal, misalnya jumlah minimum *active S-box* atau batas peluang lintasan. Jumlah 16 *round* adalah pilihan konservatif tanpa bukti keamanan.
- *Rotation* hanya memakai 4 bit dari K_r sehingga hanya ada 16 variasi rotasi per *round*.
- *Mixing* adalah transformasi linear atas bilangan bulat dengan *carry* satu arah. Perubahan pada bit paling tinggi (MSB) suatu byte masukan hanya mengubah MSB keluaran karena tidak ada *carry* yang keluar, sehingga *diffusion* bit tinggi di dalam *Mixing* lemah dan baru disebarkan oleh langkah lain.
- *Permutation* hanya mengirim *nibble* dari satu kolom ke dua kolom, sehingga pencampuran antarkolom yang penuh bertumpu pada *Rotation* dan *round* berikutnya. Kami belum memiliki batas teoretis untuk hal ini.
- Apakah kunci utama dapat dipulihkan dari satu *round key* belum dianalisis.
- Implementasi Python tidak *constant-time* dan memakai akses tabel yang bergantung pada data, sehingga rentan terhadap serangan *side-channel*. Rancangan ini ditujukan untuk tugas kuliah, bukan untuk penggunaan produksi.

IV. IMPLEMENTASI DAN SIMULASI

A. Lingkungan dan Struktur Program

Program ditulis dalam Python (target 3.13, minimal 3.12) dan antarmuka terminalnya memakai pustaka Textual. Dependensi dikelola dengan `uv`. Seluruh algoritma *block cipher*, mode operasi, *padding*, dan CMAC ditulis sendiri tanpa pustaka kriptografi pihak ketiga. Modul `secrets` dari pustaka standar hanya dipakai untuk membangkitkan IV acak dan membandingkan *tag* secara *constant-time*, sedangkan *matplotlib* hanya dipakai pada skrip pengujian untuk menggambar histogram. Program dapat dijalankan di Windows maupun Linux.

B. Antarmuka Program

Program dijalankan dengan `uv run python -m app`. Tanpa argumen (atau dengan `tui`) program membuka antarmuka terminal interaktif, sedangkan subperintah `encrypt` dan `decrypt` menyediakan antarmuka baris perintah.

Pada CLI, masukan dapat berupa berkas (`-i`) atau teks (`--text`). Opsi yang tersedia adalah `-o` (berkas ke-

Tabel 7. Struktur direktori program.

Modul	Isi
`core/state.py`	kelas `State` (16 byte), konversi byte–*nibble*, akses baris dan kolom
`core/sbox.py`, `permutation.py`	*Substitution* dan *Permutation* beserta inversnya
`core/keyExpansion.py`	*Key Schedule*, yaitu 17 *round key*
`core/cipher.py`	`TENUN128Cipher` dengan `encrypt_block`, `decrypt_block`, *Key Addition*, *Mixing*, dan *Rotation*
`modes/operations.py`, `padding.py`	mode ECB, CBC, CFB, OFB, CTR dan *padding* PKCS#7
`app/integrity.py`	CMAC berbasis TENUN-128 dan penurunan kunci MAC
`app/service.py`	format *envelope*, enkripsi dan dekripsi berkas maupun teks, verifikasi integritas
`app/cli.py`, `tui.py`, `file_picker.py`	antarmuka CLI dan TUI
`tools/sboxGenerator.py`	pencari *S-box* dengan kriteria DDT dan LAT
`tools/demo_cases.py`, `testcases/implementation/`	pembangkit dan hasil berkas uji enkripsi dan dekripsi untuk kelima mode
`tests/`	tes otomatis pytest (`test_*.py`), `test_script.py` (*avalanche*, entropi, histogram, performa), serta berkas uji dan hasilnya

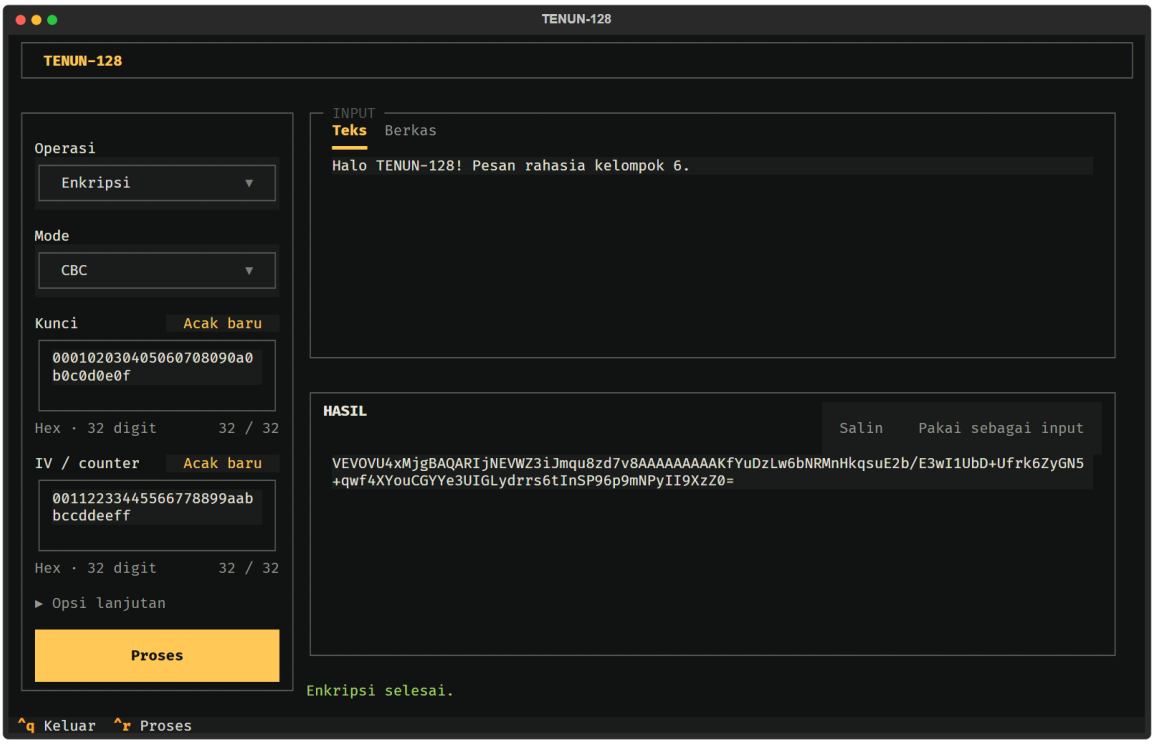
luaran baru, wajib untuk masukan berkas), `–key` (jika kosong, kunci diminta lewat *prompt* tersembunyi), `–key-format` (`hex` 32 digit atau `text` 16 byte UTF-8), `–mode` (ECB, CBC, CFB, OFB, atau CTR; bawaan CBC), dan `–iv` (32 digit heksadesimal; jika kosong, IV dibuat acak). Hasil enkripsi teks dicetak dalam Base64. Berkas keluaran dibuka dengan mode `xb` sehingga berkas yang sudah ada tidak pernah tertimpa. Berikut contoh sesi yang dihasilkan langsung dari program.

```
$ python -m app encrypt --text "Halo TENUN-128!" --mode CBC \
    --key 000102030405060708090a0b0c0d0e0f \
    --iv 00112233445566778899aabbccddeeff
VEVOVU4xMjgBAQARIjNEVWZ3iJmqu8zd7v8AAAAAAAAAD3lk31lvkHK1q97rLCve4iC5oBB+VA0FeJekkxdXJilA

$ python -m app decrypt --text <base64 di atas> \
    --key 000102030405060708090a0b0c0d0e0f
Halo TENUN-128!

$ python -m app decrypt --text <base64 dengan 1 karakter diubah> \
    --key 000102030405060708090a0b0c0d0e0f
Gagal: Verifikasi integritas gagal: kunci salah atau data berubah.   (exit code 1)
```

Pada TUI, panel kiri berisi pilihan operasi (enkripsi atau dekripsi), mode, kunci (dengan tombol *Acak baru*), IV atau *counter*, dan opsi lanjutan (format kunci dan berkas keluaran untuk hasil teks). Panel kanan berisi dua tab masukan, yaitu *Teks* dan *Berkas* (dengan pemilih berkas bawaan), serta panel hasil dengan tombol *Salin* dan *Pakai sebagai input*. Pemrosesan berjalan di *thread* terpisah sehingga tampilan tetap responsif. Saat dekripsi, mode dan IV dibaca dari *envelope* dan ditampilkan (hanya-baca), sehingga pengguna cukup memasukkan kunci. `Ctrl+R` dipakai untuk memproses dan `Ctrl+Q` untuk keluar (Gambar 7).



Gambar 7. Antarmuka terminal (TUI) program TENUN-128.

C. Implementasi Mode Operasi

Kelima mode berada dalam satu fungsi, `modes.operations._transform`, sehingga logikanya seragam. Beberapa hal yang perlu dicatat:

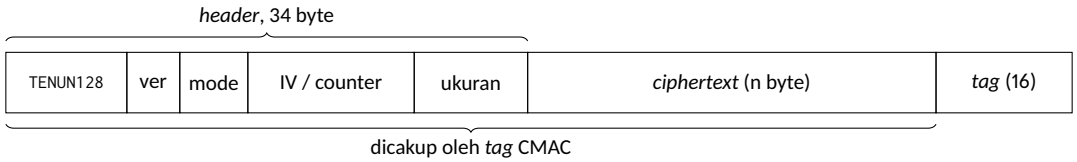
- ECB tidak memakai IV (jika diberikan, program menolak). Empat mode lainnya mewajibkan IV tepat 16 byte. Pada CTR, IV adalah nilai awal *counter* 128 bit (*big-endian*) yang ditambah 1 untuk setiap blok, dan program menolak jika *counter* akan melewati 2^{128}.
- CFB yang dipakai adalah CFB dengan segmen penuh 128 bit. Pada CFB, OFB, dan CTR, dekripsi tetap memakai fungsi *enkripsi* blok, dan blok terakhir yang pendek cukup dipotong dari *keystream*.
- ECB dan CBC memakai *padding* PKCS#7 saat enkripsi dan melepasnya saat dekripsi.
- Pada program utama, IV dibangkitkan secara acak (`secrets.token_bytes(16)`) bila pengguna tidak memberikannya, dan IV selalu disimpan di dalam *envelope*.

D. Format Berkas TENUN dan Verifikasi Integritas

Agar integritas terjamin, hasil enkripsi dibungkus dalam *envelope* berikut (Gambar 8):

$$\underbrace{\text{TENUN128}}_{8} \,\|\, \underbrace{\text{versi}}_{1} \,\|\, \underbrace{\text{id mode}}_{1} \,\|\, \underbrace{\text{IV/counter}}_{16} \,\|\, \underbrace{\text{ukuran plaintext}}_{8,\ \text{uint64 BE}} \,\|\, \underbrace{\text{ciphertext}}_{n} \,\|\, \underbrace{\text{tag}}_{16}.$$

Header berukuran 34 byte dan *tag* 16 byte, sehingga *overhead* tetapnya 50 byte, ditambah *padding* sebesar 1 sampai 16 byte pada ECB dan CBC. Untuk ECB, bidang IV diisi nol.



Gambar 8. Format berkas hasil enkripsi (lebar kolom tidak sebanding dengan ukuran byte).

Dari kunci utama K diturunkan dua kunci terpisah dengan mengenkripsi konstanta publik menggunakan TENUN-128 sendiri:

$$K_{enc} = E_K(\texttt{"TENUN-ENC-KEY-v1"}), \qquad K_{mac} = E_K(\texttt{"TENUN-MAC-KEY-v1"}).$$

Pemisahan ini mencegah satu kunci dipakai untuk dua keperluan. Saat enkripsi, *plaintext* dienkripsi dengan mode yang dipilih memakai K_{enc}, lalu *tag* dihitung dari seluruh *header* dan *ciphertext*: tag = $\text{CMAC}_{K_{mac}}$(header $\|$ ciphertext). Pola ini disebut *encrypt-then-MAC*.

CMAC diimplementasikan menurut NIST SP 800-38B [16] dengan TENUN-128 sebagai *block cipher*-nya. Subkunci dihitung sebagai $K_1 = 2 \cdot E(0^{128})$ dan $K_2 = 2 \cdot K_1$ pada GF(2^{128}) (reduksi dengan `87`). Blok akhir yang lengkap di-XOR dengan K_1, sedangkan blok akhir yang tidak lengkap, atau pesan kosong, diberi *padding* `80 00`…lalu di-XOR dengan K_2.

Fungsi `service.decrypt` memeriksa berkas dengan urutan berikut:

1. panjang berkas minimal 34 + 16 byte;
2. *tag* dihitung ulang dan dibandingkan dengan `secrets.compare_digest` *sebelum* apa pun didekripsi. Jika berbeda, program melaporkan “Verifikasi integritas gagal: kunci salah atau data berubah”;
3. *magic*, versi, id mode, dan kesesuaian ukuran *ciphertext* dengan ukuran *plaintext* pada metadata diperiksa;
4. *ciphertext* didekripsi, lalu ukuran hasilnya dicocokkan dengan ukuran *plaintext* pada metadata.

Karena *tag* diperiksa lebih dulu, *ciphertext* yang telah diubah tidak pernah sampai ke proses dekripsi maupun pelepasan *padding*, sehingga serangan *padding oracle* tidak dapat dilakukan.

E. Penanganan *Padding*

ECB dan CBC memerlukan data berkelipatan 16 byte, sedangkan CFB, OFB, dan CTR tidak. Kami memilih PKCS#7 [17] karena tiga alasan. Pertama, *padding* ini selalu dapat dibalik tanpa ambigu: byte terakhir menyatakan jumlah *padding*, dan *padding* selalu ditambahkan sehingga pesan yang sudah berkelipatan 16 byte mendapat satu blok penuh `10`×16. Pada *zero padding*, data yang berakhir dengan byte nol tidak dapat dipulihkan dengan benar. Kedua, *padding* dapat divalidasi, karena semua byte *padding* harus bernilai N. Ketiga, metodenya sederhana dan baku. Pada mode CFB, OFB, dan CTR, blok terakhir yang pendek cukup dipotong, dan panjang *plaintext* asli tetap dicatat di *header*.

Jika $N = 16 - (|P| \bmod 16)$, program menambahkan N byte bernilai N. Contohnya, “HALO!” (5 byte) menjadi `48414C4F210B0B0B0B0B0B0B0B0B0B0B` , dengan N = 11 = `0B`. Saat pelepasan, program memeriksa bahwa $1 \leq N \leq 16$ dan N byte terakhir semuanya bernilai N.

Tabel 8. Penanganan *padding* untuk berbagai panjang *plaintext* L (byte), dengan panjang yang sama seperti pada `test_roundtrip` di `tests/test_modes.py`. Kolom ukuran *ciphertext* adalah ukuran mentah (tanpa *header* dan *tag*), dan kolom terakhir adalah ukuran *envelope* untuk CBC.

L	|pad(P)|	isi *padding*	ECB dan CBC	CFB, OFB, CTR	*envelope* CBC
0	16	`10` × 16	16	0	66
1	16	`0F` × 15	16	1	66
15	16	`01` × 1	16	15	66
16	32	`10` × 16	32	16	82
17	32	`0F` × 15	32	17	82
31	32	`01` × 1	32	31	82
32	48	`10` × 16	48	32	98
33	48	`0F` × 15	48	33	98
255	256	`01` × 1	256	255	306
1024	1040	`10` × 16	1040	1024	1090

Tabel 8 menunjukkan bahwa ECB dan CBC selalu menghasilkan *ciphertext* sebesar $16(\lfloor L/16 \rfloor + 1)$ byte, termasuk untuk L = 0 dan L kelipatan 16. Mode CFB, OFB, dan CTR menghasilkan *ciphertext* yang sama panjang dengan *plaintext*. Hubungan ukuran ini diperiksa oleh `test_roundtrip`, dan tes yang sama memastikan hasil dekripsi sama dengan *plaintext* awal. *Padding* yang rusak ditolak oleh fungsi pelepasan *padding* (Tabel 9), seperti yang diperiksa oleh `test_invalid_padding`. Pada program lengkap, kerusakan seperti ini sudah lebih dulu terdeteksi oleh *tag*.

Tabel 9. Penolakan *padding* yang tidak valid oleh `unpad`.

Kasus uji	**Hasil**
Data kosong	Data PKCS#7 harus berisi blok lengkap.
1 byte (bukan kelipatan 16)	Data PKCS#7 harus berisi blok lengkap.
16 byte nol (byte terakhir `00`)	Padding PKCS#7 tidak valid.
Byte terakhir `11` (> 16)	Padding PKCS#7 tidak valid.
Padding tidak konsisten (`..0102`)	Padding PKCS#7 tidak valid.

F. Hasil Uji Enkripsi dan Dekripsi Berkas

Uji enkripsi dan dekripsi memakai berkas di folder `testcases/implementation/`, yaitu berkas teks `text.txt` (38 byte) dan berkas biner `binary.bin` (259 byte, berisi semua nilai byte 0 sampai 255 ditambah tiga byte `00`, `FF`, `00`). Keduanya dienkripsi dengan kelima mode memakai kunci `000102...0F` dan IV `101112...1F`, lalu hasilnya didekripsi dan dibandingkan byte demi byte dengan berkas asli. Berkas hasil enkripsi (`*.tenun`) dan hasil dekripsi (`*.restored`) tersimpan di folder yang sama. Hasil ringkasnya ada di `results.json` (Tabel 10). Berkas ini dapat dibangkitkan ulang dengan `tools/demo_cases.py`, dan `tests/test_demo_fixtures.py` memastikan seluruh berkas tersebut dapat direproduksi.

Tabel 10. Hasil uji enkripsi dan dekripsi pada berkas teks dan biner (dari `testcases/implementation/results.json`).

Berkas	Mode	Asli (byte)	*Envelope* (byte)	Dekripsi sama dengan asli	Perubahan ditolak
text.txt	ECB	38	98	ya	ya
text.txt	CBC	38	98	ya	ya
text.txt	CFB	38	88	ya	ya
text.txt	OFB	38	88	ya	ya
text.txt	CTR	38	88	ya	ya
binary.bin	ECB	259	322	ya	ya
binary.bin	CBC	259	322	ya	ya
binary.bin	CFB	259	309	ya	ya
binary.bin	OFB	259	309	ya	ya
binary.bin	CTR	259	309	ya	ya

Kesepuluh kombinasi berkas dan mode mengembalikan berkas asli secara identik, dan *envelope* yang diubah satu bitnya ditolak. Selisih ukuran antara *envelope* dan berkas asli pada CFB, OFB, dan CTR sebesar 50 byte, yaitu *header* 34 byte dan *tag* 16 byte. Pada ECB dan CBC ditambahkan *padding*, yaitu 10 byte untuk `text.txt` (38 menjadi 48 byte) dan 13 byte untuk `binary.bin` (259 menjadi 272 byte). Berkas biner diperlakukan sebagai deretan byte biasa sehingga tidak perlu penanganan khusus. Untuk keperluan lain, `tests/test_service.py` juga menguji berkas biner lewat `encrypt_file` dan `decrypt_file` untuk kelima mode.

G. Pengujian Integritas

Verifikasi integritas diuji oleh `tests/test_service.py` (Tabel 11). Pada tes yang paling ketat, sebuah pesan dienkripsi dengan masing-masing dari kelima mode. Setelah itu satu bit pada setiap posisi byte *envelope*, mulai dari *header*, IV, ukuran, *ciphertext*, sampai *tag*, dibalik satu per satu, dan setiap perubahan harus ditolak dengan galat `IntegrityError`. Contoh pada subbab antarmuka juga menunjukkan bahwa satu karakter Base64 yang diubah menyebabkan dekripsi ditolak.

Tabel 11. Skenario pengujian integritas pada `tests/test_service.py`. Semua skenario menghasilkan penolakan.

Skenario	Tes
1 bit pada setiap posisi byte *envelope* diubah (kelima mode)	`test_every_byte_is_authenticated`
Envelope dipotong 1 byte, ditambah 1 byte, atau dipotong menjadi 20 byte (kelima mode)	`test_every_byte_is_authenticated`
Dekripsi dengan kunci yang salah (kelima mode)	`test_every_byte_is_authenticated`
Metadata yang ber-*tag* valid tetapi isinya tidak valid: versi, id mode, atau ukuran (CBC)	`test_authenticated_but_invalid_metadata`
Berkas keluaran tidak dibuat jika dekripsi gagal, dan berkas yang sudah ada tidak tertimpa (kelima mode)	`test_binary_files_and_failed_decryption_preserves_output`
Fungsi dekripsi mentah tidak pernah dipanggil sebelum *tag* terverifikasi	`test_invalid_authentication_never_calls_decrypt`

Tanpa MAC, mode seperti CTR dan OFB bersifat *malleable*: membalik satu bit *ciphertext* membalik bit yang sama pada *plaintext*, sehingga penyerang dapat mengubah pesan secara terarah tanpa ketahuan. Pada CBC, blok yang diubah menjadi rusak dan bit yang sama pada blok berikutnya ikut terbalik. Karena *tag* CMAC diperiksa sebelum dekripsi, perubahan seperti ini ditolak.

H. Pengujian Otomatis dan Berkas Uji

Selain pengujian pada subbab-subbab sebelumnya, repositori memiliki rangkaian tes otomatis dengan pytest di folder `tests/` dan berkas uji di folder `testcases/implementation/`. Tabel 12 merangkum cakupan tiap berkas tes.

Tabel 12. Cakupan tes otomatis pada folder `tests/`.

Berkas	Yang diuji
`test_cipher.py`	enkripsi dan dekripsi satu blok mengembalikan *plaintext* semula
`test_modes.py`	*round trip* kelima mode pada sepuluh panjang data dari 0 sampai 1024 byte; hasil tiap mode dibandingkan dengan rekurensi yang ditulis ulang secara eksplisit; ECB mempertahankan pola sedangkan CBC tidak; batas *counter* CTR pada $2^{128} - 1$; IV yang tidak valid; *padding* yang tidak valid; *ciphertext* dengan blok tidak lengkap
`test_service.py`	*round trip envelope*; perubahan 1 bit pada *setiap* posisi byte *envelope* ditolak pada kelima mode; pemotongan, penambahan, dan kunci yang salah ditolak; berkas biner; berkas keluaran yang sudah ada tidak tertimpa dan berkas tidak dibuat jika dekripsi gagal; fungsi dekripsi mentah tidak dipanggil sebelum *tag* terverifikasi; metadata yang ber-*tag* valid tetapi isinya tidak valid ditolak; penguraian kunci dan IV
`test_cmac_reference.py`	CMAC diuji terhadap vektor contoh NIST SP 800-38B (AES-128) dengan panjang pesan 0, 16, 20, dan 64 byte
`test_demo_fixtures.py`	berkas di `testcases/implementation/` dapat direproduksi, dan vektor uji TENUN-128 untuk satu blok dikunci nilainya
`test_cli.py`	*round trip* teks lewat CLI untuk kelima mode, berkas biner, serta kode keluar untuk masukan yang salah
`test_tui.py`, `test_file_picker.py`	skenario TUI dan pemilih berkas: *round trip* semua mode, validasi masukan, ukuran terminal kecil, opsi lanjutan, IV dan kunci acak, tombol salin dan *Pakai sebagai input*, serta pembatalan

Pengujian CMAC memerlukan penjelasan. Karena CMAC yang diuji memakai TENUN-128, vektor NIST (yang dihitung dengan AES) tidak dapat dipakai langsung. Tes ini menggantikan *block cipher* dengan tabel keluaran AES yang dipublikasikan NIST untuk masukan-masukan yang dipakai CMAC, lalu memeriksa bahwa fungsi `cmac` menghasilkan *tag* yang sama dengan nilai resmi. Dengan cara ini, penurunan subkunci, *padding*, dan XOR blok akhir terbukti benar secara independen dari TENUN-128.

Folder `testcases/implementation/` berisi dua berkas masukan, yaitu `text.txt` (38 byte, teks UTF-8) dan `binary.bin` (259 byte, berisi semua nilai byte 0 sampai 255). Untuk setiap berkas dan setiap mode terdapat sepasang berkas hasil enkripsi (`*.tenun`) dan hasil dekripsi (`*.restored`), sehingga seluruhnya ada 20 berkas hasil. Kunci dan IV yang dipakai tetap dan bersifat publik (kunci `000102...0F`, IV `101112...1F`). Berkas `results.json` mencatat bahwa kesepuluh kasus (2 berkas $\times$ 5 mode) memulihkan berkas asli dan menolak *envelope* yang diubah. Berkas ini dibangkitkan oleh `tools/demo_cases.py` dan diperiksa ulang oleh `test_demo_fixtures.py`. Skrip `tests/test_script.py` menjalankan pengujian keamanan pada Bab V.

Seluruh rangkaian tes dijalankan dengan `uv run pytest` pada salinan bersih dari *commit* `05eb3e7` (akhir baris LF), baik di Windows maupun di Linux (WSL Ubuntu), dan menghasilkan **172 tes lulus** pada keduanya. Catatan: pada Windows dengan `core.autocrlf=true`, `text.txt` di *working tree* menjadi CRLF (40 byte), sehingga lima tes reproduksi berkas uji teks pada `test_demo_fixtures.py` gagal walaupun kodenya benar.

V. ANALISIS KEAMANAN

Semua pengujian pada bab ini dijalankan pada kelima mode operasi dan dapat diulang dari repositori. Pengujian pada berkas dilakukan oleh skrip `tests/test_script.py` (perintahnya ada di Lampiran), dengan hasil lengkap di `tests/files/result.txt` dan histogram di `tests/files/histogram.png`. Skrip ini memakai kunci `b9d1062a71a0dbdaee7b02e916ceca39`, IV `da0773423b2a300f3f75430ef76d0f7f`, dan *counter* awal `17884be59a8704dbb659d13141910cb3`. *Avalanche* pada tingkat blok dihitung dengan potongan kode kecil yang dilampirkan pada Lampiran.

A. Uji *Avalanche Effect*

1. Pada tingkat berkas

Berkas `medium.txt` (94,4 kB) dienkripsi. Setelah itu, bit ke-0 *plaintext* (atau kunci) dibalik dan berkas dienkripsi lagi dengan mode yang sama. Persentase bit *ciphertext* yang berbeda ditampilkan pada Tabel 13.

Tabel 13. *Avalanche effect* pada `medium.txt`: persentase bit *ciphertext* yang berubah.

Mode	*Flipped plaintext*	*Flipped key*
ECB	0.01%	50.05%
CBC	50.02%	49.94%
CFB	50.02%	50.01%
OFB	0.00%	50.01%
CTR	0.00%	50.00%

Perubahan 1 bit kunci mengubah sekitar 50% bit *ciphertext* pada semua mode, jadi pengaruh kunci tersebar ke seluruh keluaran. Untuk perubahan 1 bit *plaintext*, hasilnya berbeda antarmode dan perlu dibaca menurut sifat tiap mode. Perbedaan itu bukan kelemahan cipher.

- ECB (0,01%): tiap blok dienkripsi sendiri-sendiri, sehingga perubahan hanya memengaruhi satu blok (sekitar 64 dari 755.328 bit).
- OFB dan CTR (0,00%): *ciphertext* adalah *plaintext* yang di-XOR dengan *keystream* yang tidak bergantung pada *plaintext*, sehingga satu bit *plaintext* hanya mengubah satu bit *ciphertext*. Ini sifat mode yang bekerja seperti *stream cipher*, bukan sifat *block cipher*-nya.
- CBC dan CFB (50,02%): *ciphertext* blok pertama yang berubah menjadi masukan bagi blok berikutnya (*chaining* atau *feedback*), sehingga semua blok sesudahnya berubah sekitar separuh bitnya.

Karena angka pada tingkat berkas dipengaruhi oleh mode, kualitas *block cipher* sendiri perlu diukur pada satu blok.

2. Pada tingkat blok (cipher saja)

Kami membangkitkan 2000 pasangan (kunci, *plaintext*) secara acak dengan `random.Random(2026)`. Pada tiap pasangan, satu bit acak dibalik pada *plaintext* atau pada kunci, lalu dihitung persentase bit *ciphertext* 128 bit yang berubah (Tabel 14). Untuk permutasi acak yang ideal, jumlah bit yang berubah berdistribusi binomial dengan rata-rata 50% dan simpangan baku $\sqrt{128 \cdot \frac{1}{4}}/128 = 4{,}42\%$.

Tabel 14. *Avalanche effect* pada satu blok (2000 percobaan acak).

Perubahan	Rata-rata (%)	Simpangan baku (%)	Min (%)	Maks (%)
Bit *plaintext* diubah	50.15	4.40	34.38	64.06
Bit kunci diubah	49.83	4.47	36.72	66.41

Rata-ratanya mendekati 50%, dan simpangan bakunya dekat dengan nilai ideal 4,42%. Jadi pada ukuran blok, cipher berperilaku seperti permutasi acak untuk metrik ini. Hasil ini adalah bukti empiris dan bukan bukti keamanan yang formal.

B. Analisis Entropi

Entropi Shannon dihitung pada *ciphertext* dari `medium.txt` (94.401 byte) untuk setiap mode (Tabel 15).

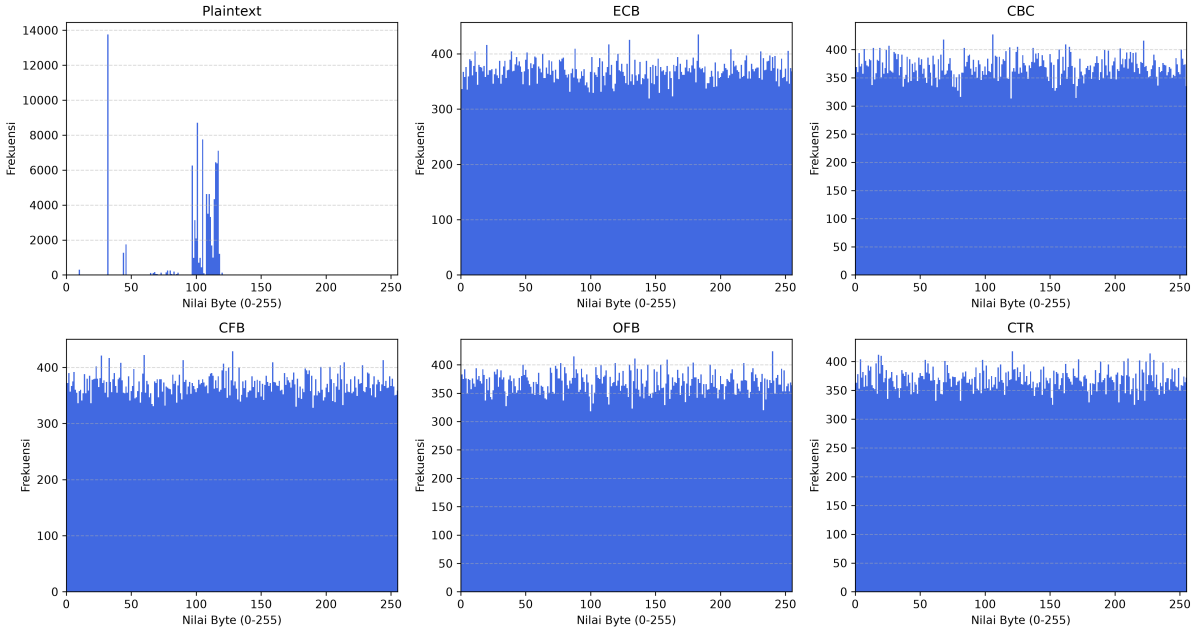
Tabel 15. Entropi *ciphertext* dari `medium.txt` (bit/byte, nilai maksimum 8).

Mode	Entropi (bit/byte)
ECB	7.998206
CBC	7.997975
CFB	7.998256
OFB	7.998162
CTR	7.998368

Semua mode menghasilkan entropi antara 7,99798 dan 7,99837 bit/byte, mendekati maksimum 8. Entropi yang dihitung dari sampel berhingga selalu sedikit di bawah 8. Koreksi bias Miller–Madow memberikan nilai harapan $8 - \frac{255}{2n \ln 2} \approx 7{,}99805$ untuk n = 94.401 byte, dan nilai yang terukur sesuai dengan itu. Jadi tidak ada penyimpangan keacakan byte yang terlihat pada kelima mode.

C. Analisis Histogram

Gambar 9 membandingkan histogram frekuensi byte *plaintext* `medium.txt` dengan *ciphertext* dari tiap mode.



Gambar 9. Histogram frekuensi byte: *plaintext* dan *ciphertext* (ECB, CBC, CFB, OFB, CTR) untuk `medium.txt`.

Histogram *plaintext* sangat tidak seragam. Nilai byte terkonsentrasi pada karakter ASCII, dengan puncak pada spasi (nilai 32, sekitar 13.700 kemunculan) dan kelompok huruf kecil (nilai 97 sampai 122), sedangkan sebagian besar nilai 0 sampai 255 tidak pernah muncul. Histogram *ciphertext* pada kelima mode menyebar ke seluruh 256 nilai byte dengan frekuensi sekitar 320 sampai 430 di sekitar rata-rata 369 (94.401/256). Simpangan sekitar ± 50 sesuai dengan fluktuasi acak (simpangan baku Poisson sekitar 19 untuk 256 kelas). Jadi *ciphertext* mendekati distribusi seragam dan pola frekuensi huruf pada *plaintext* tidak terlihat lagi.

Histogram dan entropi hanya melihat frekuensi byte, sehingga keduanya tidak bisa membedakan ECB dari mode lain. Kelemahan ECB terlihat pada `test_ecb_repetition_and_cbc_chaining` di `tests/test_modes.py`: untuk *plaintext* berupa dua blok yang identik (32 byte nol), dua blok *ciphertext* pada ECB sama persis, sedangkan pada CBC berbeda. Karena itu program memakai CBC sebagai mode bawaan, dan ECB sebaiknya hanya dipakai untuk satu blok atau untuk pengujian.

D. Uji Performa

Waktu enkripsi dan dekripsi diukur pada tiga berkas teks dengan ukuran berbeda, yaitu `small.txt` (559 byte), `medium.txt` (94401 byte), dan `big.txt` (10921739 byte). Pengukuran memakai mode mentah tanpa *envelope*.

Tabel 16. Hasil uji performa untuk tiap mode dan ukuran berkas.

Mode	Berkas	Enkripsi (ms)	Dekripsi (ms)	*Throughput* enkripsi (MB/s)	*Throughput* dekripsi (MB/s)
ECB	small.txt	6.649	6.638	0.08	0.08
	medium.txt	1116.468	1112.181	0.08	0.08
	big.txt	129210.206	129839.829	0.08	0.08
CBC	small.txt	6.617	6.654	0.08	0.08
	medium.txt	1124.498	1120.518	0.08	0.08
	big.txt	130119.328	131346.065	0.08	0.08
CFB	small.txt	6.971	6.785	0.08	0.08
	medium.txt	1145.950	1162.650	0.08	0.08
	big.txt	133394.689	132138.895	0.08	0.08
OFB	small.txt	6.657	6.683	0.08	0.08
	medium.txt	1127.070	1123.406	0.08	0.08
	big.txt	130242.275	129958.805	0.08	0.08
CTR	small.txt	6.814	6.636	0.08	0.08
	medium.txt	1151.491	1126.123	0.08	0.08
	big.txt	130564.144	130350.889	0.08	0.08

Dari tabel terlihat beberapa hal. Pertama, waktu naik secara linear terhadap ukuran berkas: rasio waktu `big.txt` terhadap `medium.txt` sekitar 116, sama dengan rasio ukurannya. Kedua, *throughput* sekitar 0,08 MB/s untuk semua mode, atau sekitar 0,19 ms per blok (`big.txt` pada ECB butuh 129,2 detik untuk 682.609 blok). Ketiga, selisih antarmode sekitar 3% atau kurang, karena biaya hampir seluruhnya berasal dari satu pemanggilan fungsi blok per blok data, sedangkan XOR dan *chaining* nyaris tidak berpengaruh. Keempat, waktu dekripsi hampir sama dengan enkripsi (selisih di bawah 2%) karena operasi baliknya sama mahal, dan pada CFB, OFB, dan CTR keduanya memang memakai fungsi enkripsi blok.

Throughput yang rendah berasal dari implementasi Python murni. Setiap langkah membuat objek `State` baru dan mengonversi antara `bytes`, daftar *nibble*, dan `bytearray`. Hal ini bukan sifat algoritmanya, dan saran optimasinya ada di Bab VI. Pada program lengkap, *envelope* menambah satu lintasan CMAC, yaitu satu pemanggilan fungsi blok lagi per blok data, sehingga waktu sebenarnya kira-kira dua kali lipat angka pada Tabel 16. Angka waktu absolut bergantung pada perangkat keras.

E. Pengujian Tambahan

S-box yang dipakai memiliki DU = 4 dan LU = 4 (nonlinearitas 4) dan tidak punya *fixed point*, sesuai kriteria pada Bab III. Nilai ini sama dengan batas terbaik untuk *S-box* 4 bit yang bijektif. Artinya, peluang diferensial maksimum untuk satu *S-box* yang aktif adalah $4/16 = 2^{-2}$, dan bias linear maksimumnya juga $4/16 = 2^{-2}$. Nilai-nilai ini dapat dihitung ulang dengan fungsi pada `tools/sboxGenerator.py` (Lampiran).

Pengujian integritas dan *padding* pada Bab IV serta tes otomatis pada subbab IV.H melengkapi analisis ini. Khususnya, perubahan 1 bit pada setiap posisi byte *envelope* ditolak pada kelima mode.

F. Ringkasan dan Keterbatasan

Hasil pengujian menunjukkan bahwa TENUN-128 memenuhi indikator statistik yang diminta. *Avalanche* pada tingkat blok mendekati 50% untuk perubahan *plaintext* maupun kunci, entropi *ciphertext* mendekati 8 bit/byte, dan histogram mendekati seragam. Meski begitu, uji statistik seperti ini tidak membuktikan keamanan. Kami belum melakukan kriptanalisis diferensial dan linear (misalnya menghitung jumlah minimum *active S-box*), analisis *related-key* dan *slide attack*, maupun analisis aljabar. Jumlah 16 *round* ditentukan secara konservatif dan tidak berasal dari batas keamanan yang terbukti. Implementasinya juga tidak dirancang tahan terhadap serangan *side-channel*. Cipher ini sebaiknya dipandang sebagai rancangan akademik.

VI. KESIMPULAN DAN SARAN PENGEMBANGAN

A. Kesimpulan

1. Kami telah merancang dan mengimplementasikan *block cipher* TENUN-128, yaitu SPN dengan blok dan kunci 128 bit serta 16 *round* ditambah *whitening* di akhir. Setiap *round* terdiri atas *Key Addition* (XOR kunci), *Substitution* (*S-box* 4 bit dengan DU = LU = 4 dan tanpa *fixed point*), *Permutation* (permutasi *nibble* j = (9i + 5) mod 32), *Mixing* (rantai penjumlahan modulo 2^8 pada tiap kolom, dengan matriks bulat berdeterminan 1), dan *Rotation* (rotasi *word* 32 bit pada tiap baris yang bergantung pada kunci). *Key Schedule* membangkitkan *round key* dengan konstanta dari rasio emas.
2. Program mendukung teks dan berkas biner pada lima mode operasi (ECB, CBC, CFB, OFB, CTR), *padding* PKCS#7, dan jaminan integritas lewat *envelope* serta CMAC berbasis TENUN-128 dengan pola *encrypt-then-MAC*. Semua uji enkripsi dan dekripsi pada berkas teks dan biner mengembalikan berkas asli secara identik, dan perubahan 1 bit pada setiap posisi byte *envelope* ditolak. Seluruh 172 tes otomatis pada repositori lulus (Bab IV, Subbab H).
3. Hasil analisis menunjukkan *avalanche* pada tingkat blok sekitar 50% untuk perubahan 1 bit *plaintext* maupun kunci, dengan simpangan baku yang sesuai nilai ideal. Entropi *ciphertext* sebesar 7,998 bit/byte pada semua mode, dan histogram *ciphertext* mendekati seragam. Perbedaan *avalanche* pada tingkat berkas antarmode, misalnya 0,00% pada OFB dan CTR, berasal dari sifat mode dan bukan dari kelemahan cipher.
4. Histogram dan entropi tidak dapat mendeteksi kebocoran pola pada ECB. Tes `test_ecb_repetition_and_cbc_chaining` menunjukkan bahwa ECB tetap membocorkan keberulangan blok, sehingga CBC dipilih sebagai mode bawaan.
5. *Throughput* implementasi sangat rendah (sekitar 0,08 MB/s) karena memakai Python murni, tetapi waktunya linear terhadap ukuran data dan hampir sama pada semua mode.
6. Keamanan TENUN-128 baru diuji secara empiris dan belum ada kriptanalisis formal, sehingga cipher ini adalah rancangan akademik dan bukan pengganti cipher standar.

B. Saran Pengembangan

- Melakukan kriptanalisis formal, yaitu menghitung jumlah minimum *active S-box* pada lintasan diferensial dan linear, menganalisis serangan *related-key* dan *slide*, serta memeriksa apakah *Key Schedule* dapat dibalik untuk memulihkan kunci utama dari satu *round key*.
- Memperbaiki *diffusion*, misalnya dengan permutasi *Permutation* yang menyebarkan *nibble* satu kolom ke keempat kolom, dan dengan memperkuat *diffusion* bit tinggi pada *Mixing*. *Rotation* juga

dapat memakai lebih dari 4 bit kunci.

- Meningkatkan performa dengan tabel pencarian yang menggabungkan *Substitution* dan *Permutation*, bekerja langsung pada `bytearray` tanpa membuat objek `State` baru, memproses seluruh blok dalam satu lintasan, atau menulis inti cipher dengan ekstensi C atau bahasa terkompilasi. Jumlah rotasi pada *Rotation* juga dapat dihitung sekali untuk setiap *round key*.
- Menambahkan fungsi penurunan kunci dari kata sandi (misalnya PBKDF2 atau scrypt dengan *salt*), karena kunci teks saat ini harus tepat 16 byte. Ukuran kunci 192 dan 256 bit juga dapat didukung.
- Memproses berkas besar secara *streaming* dengan format berpotongan yang tiap potongannya diautentikasi, karena saat ini seluruh berkas dibaca ke memori.
- Mencegah pemakaian ulang IV atau *counter* dengan kunci yang sama pada CTR dan OFB, karena pengulangan membocorkan XOR antar-*plaintext*.
- Menambah uji otomatis dengan pytest (vektor uji, *round trip* acak untuk semua mode dan panjang data, serta uji penolakan perubahan), dan analisis kerandoman lanjutan seperti NIST SP 800-22.
- Menyediakan dokumentasi API yang dihosting publik dan video demo (bonus tugas).

VII. DAFTAR PUSTAKA

PUSTAKA

[1] H. M. Heys, "A Tutorial on the Implementation of Block Ciphers: Software and Hardware Applications," Cryptology ePrint Archive, Report 2020/1545, 2020. [Online]. Available: https://eprint.iacr.org/2020/1545.pdf

[2] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," Federal Information Processing Standards Publication (FIPS) 197, NIST FIPS 197-upd1, updated May 9, 2023. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf

[3] R. L. Rivest, M. J. B. Robshaw, R. Sidney, and Y. L. Yin, "The RC6 Block Cipher," ver. 1.1, Aug. 20, 1998. [Online]. Available: https://people.csail.mit.edu/rivest/pubs/RRSY98.pdf

[4] B. Schneier, J. Kelsey, D. Whiting, D. Wagner, C. Hall, and N. Ferguson, "The Twofish Encryption Algorithm," Sep. 2000. [Online]. Available: https://www.researchgate.net/publication/234781510_The_Twofish_Encryption_Algorithm

[5] D. Hong, J. Sung, S. Hong, J. Lim, S. Lee, B.-S. Koo, C. Lee, D. Chang, J. Lee, K. Jeong, H. Kim, J. Kim, and S. Chee, "HIGHT: A New Block Cipher Suitable for Low-Resource Device," in *Cryptographic Hardware and Embedded Systems – CHES 2006*, Lecture Notes in Computer Science, vol. 4249, pp. 46–59, Springer, 2006. [Online]. Available: https://link.springer.com/chapter/10.1007/11894063_4

[6] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. B. Robshaw, Y. Seurin, and C. Vikkelsoe, "PRESENT: An Ultra-Lightweight Block Cipher," in *Cryptographic Hardware and Embedded Systems – CHES 2007*, Lecture Notes in Computer Science, vol. 4727, pp. 450–466, Springer, 2007. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-540-74735-2_31

[7] Z. Gong, S. Nikova, and Y. W. Law, "KLEIN: A New Family of Lightweight Block Ciphers," in *RFID. Security and Privacy – 7th International Workshop, RFIDSec 2011*, Lecture Notes in Computer Science, vol. 7055, pp. 1–18, Springer, 2011. [Online]. Available: https://ris.utwente.nl/ws/files/5095833/The_KLEIN_Block_Cipher.pdf

[8] T. Shirai, K. Shibutani, T. Akishita, S. Moriai, and T. Iwata, "The 128-Bit Blockcipher CLEFIA (Extended Abstract)," in *Fast Software Encryption – FSE 2007*, Lecture Notes in Computer Science, vol. 4593, pp. 181–195, Springer, 2007. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-540-74619-5_12

[9] R. M. Needham and D. J. Wheeler, "Tea Extensions," Technical Report, Computer Laboratory, University of Cambridge, Oct. 1997. [Online]. Available: http://www.club.cc.cmu.edu/~ajo/docs/xtea.pdf

[10] S. Banik, Z. Bao, T. Isobe, H. Kubo, F. Liu, K. Minematsu, K. Sakamoto, N. Shibata, and M. Shigeri, "WARP: Revisiting GFN for Lightweight 128-Bit Block Cipher," in *Selected Areas in Cryptography – SAC 2020*, Lecture

Notes in Computer Science, vol. 12804, pp. 535–564, Springer, 2021. [Online]. Available: https://eprint.iacr.org/2020/1320.pdf. doi: 10.1007/978-3-030-81652-0_21.

[11] C. Burwick, D. Coppersmith, E. D'Avignon, R. Gennaro, S. Halevi, C. Jutla, S. M. Matyas Jr., L. O'Connor, M. Peyravian, D. Safford, and N. Zunic, "MARS – A Candidate Cipher for AES," IBM Corporation, revised Sep. 22, 1999. [Online]. Available: https://shaih.github.io/pubs/mars/mars.pdf

[12] C. E. Shannon, "Communication Theory of Secrecy Systems," *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, 1949.

[13] National Institute of Standards and Technology, "Data Encryption Standard (DES)," FIPS PUB 46-3, Oct. 1999 (ditarik 2005).

[14] R. Anderson, E. Biham, and L. Knudsen, "Serpent: A Proposal for the Advanced Encryption Standard," NIST AES Proposal, 1998.

[15] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Methods and Techniques," NIST Special Publication 800-38A, Dec. 2001.

[16] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: The CMAC Mode for Authentication," NIST Special Publication 800-38B, May 2005.

[17] R. Housley, "Cryptographic Message Syntax (CMS)," RFC 5652, Sep. 2009.

[18] M. Bellare and C. Namprempre, "Authenticated Encryption: Relationships among Notions and Analysis of the Generic Composition Paradigm," in *Advances in Cryptology – ASIACRYPT 2000*, Lecture Notes in Computer Science, vol. 1976, pp. 531–545, Springer, 2000.

[19] E. Biham and A. Shamir, "Differential Cryptanalysis of DES-like Cryptosystems," *Journal of Cryptology*, vol. 4, no. 1, pp. 3–72, 1991.

[20] M. Matsui, "Linear Cryptanalysis Method for DES Cipher," in *Advances in Cryptology – EUROCRYPT '93*, Lecture Notes in Computer Science, vol. 765, pp. 386–397, Springer, 1994.

[21] H. Feistel, "Cryptography and Computer Privacy," *Scientific American*, vol. 228, no. 5, pp. 15–23, May 1973.

[22] A. F. Webster and S. E. Tavares, "On the Design of S-Boxes," in *Advances in Cryptology – CRYPTO '85*, Lecture Notes in Computer Science, vol. 218, pp. 523–534, Springer, 1986.

VIII. LAMPIRAN

A. Pranala Repository

Kode sumber, laporan, berkas uji, dan hasil pengujian:
https://github.com/AgungLucker/Tugas-2-IF4020-Kriptograf-Kelompok-6

B. Tabel Pembagian Tugas

NIM	Nama	Tugas
13523023	Muhammad Aufa Farabi	*Core cipher* dan *key schedule*: desain algoritma, *S-box*, permutasi, fungsi *round*, *key schedule*, serta enkripsi dan dekripsi blok
13523075	Muhammad Dicky Isra	Implementasi dan mode operasi: penerapan *core cipher* ke dalam program, mode ECB, CBC, CFB, OFB, dan CTR, *padding*, serta integritas
13523111	Muhammad Iqbal Haidar	Pengujian dan analisis keamanan: vektor uji, pengujian kebenaran, *avalanche*, entropi, histogram, dan pengujian semua mode
13523128	Andi Farhan Hidayat	*Research*, dokumentasi, dan integrasi: *related works*, dasar teori, diagram rancangan, laporan, README, integrasi repositori, dan demo

C. Cara Menjalankan Program dan Pengujian

Semua perintah dijalankan dari direktori proyek.

```
uv sync --locked                                    # pasang dependensi
uv run python -m app                                # antarmuka TUI
uv run python -m app --help                         # bantuan CLI
uv run pytest -q                                    # seluruh tes otomatis
uv run python -u tests/test_script.py | tee tests/files/result.txt
uv run python -m tools.demo_cases --output <folder baru>
```

Perintah terakhir membangkitkan ulang berkas uji seperti pada `testcases/implementation/` ke folder baru. Contoh sesi CLI pada Subbab IV.B juga dapat dijalankan ulang persis seperti yang tertulis.

D. Vektor Uji

Semua nilai dalam heksadesimal. V1 adalah contoh pada Subbab III.H dan dapat direproduksi dengan potongan kode di bawah. V2 dikunci nilainya oleh `test_pinned_existing_core_vector` pada `tests/test_demo_fixtures.py`.

Tabel 17. Vektor uji TENUN-128 untuk satu blok.

	Kunci	***Plaintext***	***Ciphertext***
V1	`000102030405060708090A0B0C0D0E0F`	`4C4F52454D20495053554D205349414D`	`43D34D50CEC3A30EFC2CC3D8C50573FE`
V2	`000102030405060708090A0B0C0D0E0F`	`00112233445566778899AABBCCDDEEFF`	`23C65174394040E93EE91D5C309D112B`

E. *Round Key* untuk Contoh Uji

Round key untuk K = `000102030405060708090A0B0C0D0E0F` (keluaran `TENUN128Cipher(key).key_expansion.round_keys`).

Tabel 18. *Round key* hasil *Key Schedule*.

	Nilai (heksadesimal)
K_1	`41CB7FBEDEB6E522E4F919B452AD0F47`
K_2	`4EEC06B59E7723D444E9BC9ED777FBA0`
K_3	`08223B77E24711092BF990AA67E804E6`
K_4	`7ECFFF9BFB24B77EB18B97ABA3CC9ADC`
K_5	`881E78D6CD68F370943E899B6B439276`
K_6	`A790AA8008488FFF6BBDF3E5352D064A`
K_7	`CC097E8A96DCE958A000DF9ADA644D9C`
K_8	`CF12D957C93C0CC9F2191866A4AD87DC`
K_9	`D12C421F067A9CCECCE70E4983990899`
K_{10}	`B12E33BD3B3EE19693DAC24002ADD528`
K_{11}	`3AEEF933537F4CB8F6137EBC45B772CB`
K_{12}	`D85722A25875503D793CB1FEB44A52C2`
K_{13}	`8964E394777EDB3BF8F41DFE4E210480`
K_{14}	`352534BFA4B38A3AF2EA7E7A1C551E1A`
K_{15}	`D23A2D2DF678CC79BC57863977BEA13E`
K_{16}	`8CD5E5B3EA0BB9261AE3E3E97CC9A206`
K_{17}	`A9AA1614A1FDB82CD7399075B9A4A0F8`

F. Potongan Kode untuk Mereproduksi Contoh dan Analisis

Potongan kode berikut dijalankan dari direktori proyek, misalnya dengan menempelkannya ke `uv run python`. Potongan pertama mencetak *state* pada tiap langkah di *round* pertama (Gambar 6) dan *ciphertext* akhir.

```
from core.cipher import TENUN128Cipher
from core.state import State

key, pt = bytes(range(16)), b"LOREM IPSUM SIAM"
c = TENUN128Cipher(key)
rk = c.key_expansion.round_keys
print("K1 =", rk[0].hex().upper())
s = State(pt)
langkah = [
    ("Key Addition", lambda s: c.XOR_round_key(s, rk[0])),
    ("Substitution", c.sbox.substitute_state),
    ("Permutation", c.permutation.permute_state),
    ("Mixing", c.modulo_opr),
    ("Rotation", lambda s: c.rotate_opr(s, rk[0])),
]
for nama, f in langkah:
    s = f(s)
    print(nama, s.data.hex().upper())
print("C =", c.encrypt_block(pt).hex().upper())
```

Potongan kedua menghitung *avalanche* pada tingkat blok (Tabel 14).

```
import random
import statistics
from core.cipher import TENUN128Cipher

rng = random.Random(2026)
rb = lambda n: bytes(rng.randrange(256) for _ in range(n))
hd = lambda x, y: sum((p ^ q).bit_count() for p, q in zip(x, y))

def flip(data, bit):
    a = bytearray(data)
    a[bit // 8] ^= 1 << (bit % 8)
    return bytes(a)

pt, ky = [], []
for _ in range(2000):
    k, p = rb(16), rb(16)
    c0 = TENUN128Cipher(k).encrypt_block(p)
    pt.append(100 * hd(c0, TENUN128Cipher(k).encrypt_block(flip(p, rng.randrange(128)))) / 128)
    ky.append(100 * hd(c0, TENUN128Cipher(flip(k, rng.randrange(128))).encrypt_block(p)) / 128)
for nama, x in (("plaintext", pt), ("kunci", ky)):
    print(nama, round(statistics.mean(x), 2), round(statistics.pstdev(x), 2),
          round(min(x), 2), round(max(x), 2))
```

Potongan ketiga menghitung DU, LU, dan jumlah *fixed point S-box*, serta tabel DDT dan LAT di bawah.

```
from core.sbox import SBox
from tools.sboxGenerator import compute_ddt, compute_lat, differential_uniformity, linear_uniformity

print("DU =", differential_uniformity(SBox.SBOX))
print("LU =", linear_uniformity(SBox.SBOX))
print("titik tetap =", sum(SBox.SBOX[x] == x for x in range(16)))
ddt, lat = compute_ddt(SBox.SBOX), compute_lat(SBox.SBOX)
```

G. Balikan *Permutation*

Tabel 19. *Permutation*$^{-1}$: *nibble* pada posisi j berasal dari posisi i = 25(j − 5) mod 32.

j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
i	3	28	21	14	7	0	25	18	11	4	29	22	15	8	1	26
j	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
i	19	12	5	30	23	16	9	2	27	20	13	6	31	24	17	10

H. DDT dan LAT dari *S-box*

Entri bernilai nol ditulis "·". Baris menyatakan Δx (DDT) atau a (LAT), dan kolom menyatakan Δy atau b.

Tabel 20. *Differential Distribution Table*.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	16	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·
1	·	·	·	4	·	2	2	·	·	2	·	2	·	·	2	2
2	·	2	·	2	·	·	4	·	·	·	4	·	·	2	·	2
3	·	·	2	2	·	·	·	·	2	2	·	·	2	2	2	2
4	·	·	·	·	·	·	·	4	4	2	·	2	·	2	·	2
5	·	2	·	·	2	·	2	2	2	·	·	·	2	·	4	·
6	·	2	2	·	2	·	·	2	·	2	2	·	·	2	2	·
7	·	2	·	·	4	2	·	4	·	·	2	·	·	·	2	·
8	·	·	4	·	2	2	4	·	·	·	·	·	2	2	·	·
9	·	·	2	·	·	4	·	2	·	2	·	4	2	·	·	·
A	·	·	·	·	4	·	2	2	·	2	·	2	2	·	·	2
B	·	2	·	·	2	·	·	·	2	·	4	·	·	2	·	4
C	·	2	·	·	·	2	·	·	·	2	2	2	2	4	·	·
D	·	·	2	4	·	2	·	·	2	·	2	2	·	·	2	·
E	·	2	2	2	·	·	2	·	·	·	·	2	2	·	2	2
F	·	2	2	2	·	2	·	·	4	2	·	·	2	·	·	·

Tabel 21. *Linear Approximation Table*.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	8	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·
1	·	-2	-2	·	·	-2	-2	·	2	-4	·	-2	-2	·	4	2
2	·	·	-2	-2	-2	2	·	-4	-2	-2	·	·	·	4	-2	2
3	·	2	4	2	-2	4	-2	·	·	-2	·	2	-2	·	2	·
4	·	2	·	2	·	-2	·	-2	-2	4	2	·	-2	·	2	4
5	·	4	-2	-2	·	·	-2	2	-4	·	-2	-2	·	·	2	-2
6	·	-2	2	·	2	·	·	-2	-4	-2	-2	·	2	-4	·	2
7	·	4	·	·	2	-2	-2	-2	2	-2	2	2	4	·	·	·
8	·	-2	·	-2	·	-2	-4	2	-2	·	2	4	-2	·	-2	·
9	·	·	-2	2	4	4	-2	2	·	·	2	-2	·	·	-2	2
A	·	2	2	-4	2	·	4	2	·	-2	2	·	-2	·	·	2
B	·	·	·	-4	-2	2	-2	-2	2	2	2	-2	·	-4	·	·
C	·	·	·	·	4	·	·	-4	·	·	·	·	-4	·	·	-4
D	·	-2	-2	·	·	2	2	·	-2	·	4	2	2	·	4	-2
E	·	·	-2	-2	2	2	·	·	2	2	-4	4	·	·	2	2
F	·	2	-4	2	-2	·	2	·	·	-2	·	2	-2	-4	-2	·